

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH
—o0o—



NHẬP MÔN TRÍ TUỆ NHÂN TẠO

ASSIGNMENT 1 SOKOBAN OF MAGIC

GVHD: ThS. Vương Bá Thịnh
Lớp: L01
Nhóm: ALPHA TEAM
SV thực hiện: Lê Thành Đạt - 2210683
Đình Công Minh - 2212027
Lê Công Minh - 2212044
Huỳnh Đức Nguyên - 2252542
Nguyễn Gia Thịnh - 2213286

Tp. Hồ Chí Minh, Tháng 10/2025

Mục lục

1	Giới thiệu	2
1.1	Tổng quát về đề tài	2
1.2	Tổng quát về thuật toán	2
1.2.1	Thuật toán Depth-First Search (DFS)	2
1.2.2	Thuật toán A-Star (A^*)	2
2	DFS	4
2.1	Khái niệm	4
2.2	Chiến lược tìm kiếm	5
3	A^* (A-star)	6
3.1	Khái niệm	6
3.2	Chiến lược tìm kiếm	7
4	Hệ thống trò chơi	8
4.1	Các thư viện được áp dụng	8
4.2	Áp dụng thuật toán	8
4.2.1	DFS	8
4.2.2	A^*	10
4.3	Trình bày về thư mục code và cách chạy trò chơi	12
4.3.1	Trình bày thư mục code	12
4.3.2	Hướng dẫn chạy trò chơi	13
5	Phát triển và đánh giá	19
5.1	Kết quả	19
5.2	Đối với DFS:	22
5.2.1	Đối với A^* :	23
6	Kết luận	24
7	Tài liệu tham khảo	25

1 Giới thiệu

1.1 Tổng quát về đề tài

Sokoban là một trò chơi giải đố cổ điển, trong đó người chơi phải đẩy các hộp đến các vị trí đích đã được chỉ định trong một không gian giới hạn. Điểm đặc trưng của trò chơi là người chơi chỉ có thể **đẩy** (không thể kéo) một hộp tại một thời điểm và không thể đẩy hai hộp cùng lúc. Mặc dù quy tắc đơn giản, nhưng vì số lượng trạng thái có thể tăng theo cấp số nhân với kích thước của bản đồ và số lượng hộp, việc tìm kiếm giải pháp tối ưu (số lần đẩy hoặc số bước di chuyển ít nhất) yêu cầu các thuật toán tìm kiếm thông minh. Mục tiêu chính của đề tài này là ứng dụng các thuật toán tìm kiếm có thông tin (A^*) và không thông tin (DFS) để giải quyết bài toán Sokoban để đánh giá hiệu suất của chúng trên các cấp độ khác nhau.

1.2 Tổng quát về thuật toán

Đề tài tập trung vào việc áp dụng hai thuật toán tìm kiếm chính: Depth-First Search (DFS) và A-Star (A^*). Cả hai thuật toán này đều hoạt động dựa trên việc duyệt qua không gian trạng thái của trò chơi.

1.2.1 Thuật toán Depth-First Search (DFS)

DFS là một thuật toán tìm kiếm *không thông tin* (uninformed search).

- Nguyên tắc: Thuật toán duyệt sâu nhất có thể dọc theo mỗi nhánh trước khi quay lui (backtrack). Nó sử dụng cấu trúc dữ liệu *Stack* để lưu trữ các trạng thái cần khám phá.
- Ưu điểm: Tiết kiệm bộ nhớ vì nó chỉ lưu trữ đường đi hiện tại từ gốc đến nút hiện tại. Dễ triển khai.
- Nhược điểm:
 - Không đảm bảo tối ưu: Nó có thể tìm thấy một giải pháp (đường đi) rất dài nếu giải pháp đó nằm sâu trong một nhánh được khám phá sớm.
 - Không đảm bảo đầy đủ: Trên một không gian trạng thái vô hạn hoặc có chu trình, DFS có thể bị mắc kẹt vô hạn trong một nhánh.

1.2.2 Thuật toán A-Star (A^*)

A^* là một thuật toán tìm kiếm *có thông tin* (informed search) và là một trong những thuật toán tìm đường đi phổ biến nhất.

- Nguyên tắc: A^* tìm kiếm đường đi từ nút gốc đến nút đích bằng cách duy trì một hàm đánh giá ($f(n)$) cho mỗi nút n trong không gian trạng thái. Hàm đánh giá được xác định bởi công thức:

$$f(n) = g(n) + h(n)$$

Trong đó:

- $g(n)$: Chi phí thực tế từ trạng thái ban đầu đến trạng thái hiện tại n . Trong Sokoban, đây thường là tổng số lần đẩy hộp đã thực hiện.
- $h(n)$: Hàm heuristic, là ước tính chi phí từ trạng thái hiện tại n đến trạng thái đích. Trong Sokoban, một heuristic phổ biến là tổng khoảng cách Manhattan nhỏ nhất từ mỗi hộp chưa đặt đến vị trí đích gần nhất chưa được chiếm.
- Ưu điểm:
 - Đảm bảo tối ưu: Nếu hàm heuristic $h(n)$ là hợp lệ, A^* được đảm bảo sẽ tìm thấy lời giải tối ưu (đường đi có chi phí thấp nhất).
 - Hiệu quả: A^* tập trung tìm kiếm vào các khu vực hứa hẹn nhất của không gian trạng thái, giảm thiểu đáng kể số lượng trạng thái phải xem xét so với DFS.
- Nhược điểm: Cần nhiều bộ nhớ hơn DFS vì nó cần lưu trữ tất cả các nút đã được khám phá và các nút chờ khám phá.

2 DFS

2.1 Khái niệm

DFS (Depth First Search) là một thuật toán tìm kiếm theo chiều sâu, nó duyệt theo nhánh cho đến khi không thể đi sâu hơn được nữa thì mới quay lại để duyệt các nhánh khác. Trong Sokoban, DFS có thể được sử dụng để tìm một đường đi từ vị trí ban đầu đến mục tiêu. Tuy nhiên, DFS không đảm bảo tìm được đường đi ngắn nhất, và trong một số trường hợp có thể bị rơi vào vòng lặp nếu không có cơ chế đánh dấu trạng thái đã thăm hoặc là không có giới hạn độ sâu của nhánh có thể dẫn đến sự vô hạn.

- Để sử dụng DFS trong Sokoban, bạn bắt đầu từ vị trí ban đầu của người chơi và đi sâu theo một trạng thái con.
- Từ trạng thái hiện tại, bạn tiếp tục duyệt các trạng thái con theo một nhánh cho đến khi gặp mục tiêu hoặc không còn trạng thái mới để đi.
- Nếu chưa đạt mục tiêu, chúng ta tiếp tục quay lui lại để thử ở các nhánh khác.
- Quá trình này lặp lại cho đến khi tìm thấy mục tiêu hoặc duyệt hết không gian trạng thái mà vẫn không có lời giải.

Thuật toán DFS (Depth-First Search) là một phương pháp tìm kiếm trong đồ thị, được sử dụng để duyệt qua tất cả các đỉnh và cạnh theo một trình tự có hệ thống. Thuật toán này bắt đầu từ một đỉnh xuất phát và đi sâu xuống các đỉnh kề cho đến khi không còn khả năng mở rộng, sau đó quay ngược lại (backtrack) để tiếp tục với những đỉnh chưa được duyệt.

Một mô tả chi tiết của thuật toán DFS như sau:

- Khởi tạo: Chọn một đỉnh xuất phát (thường là đỉnh đầu tiên) và đánh dấu nó là đã được thăm.
- Duyệt: Với các đỉnh kề chưa được thăm, chọn một đỉnh và lặp lại quá trình duyệt theo chiều sâu.
- Quay lui: Khi từ một đỉnh không còn đỉnh kề nào chưa được thăm, thuật toán quay lại đỉnh trước đó để tiếp tục.
- Kết thúc: Thuật toán dừng lại khi toàn bộ các đỉnh trong đồ thị đã được duyệt qua.

2.2 Chiến lược tìm kiếm

DFS là một thuật toán tìm kiếm không có trọng số, hoạt động theo nguyên tắc đi sâu nhất có thể trước khi quay lại để duyệt các nhánh khác. Trong Sokoban, DFS được sử dụng để tìm một chuỗi hành động giúp người chơi di chuyển từ vị trí ban đầu đến mục tiêu. Tuy nhiên, vì không xét đến chi phí di chuyển nên DFS không đảm bảo tìm được đường đi ngắn nhất và có thể dễ rơi vào các nhánh không có lời giải.

Để sử dụng DFS trong Sokoban, ta bắt đầu từ trạng thái ban đầu của người chơi và các hộp, sau đó duyệt theo chiều sâu bằng cách mở rộng từng hành động hợp lệ. Mỗi trạng thái mới sẽ được lưu vào ngăn xếp (stack) để tiếp tục khám phá. Thuật toán dừng lại khi tất cả các hộp được đẩy đến đúng vị trí mục tiêu hoặc khi không còn trạng thái nào để duyệt.

3 A* (A-star)

3.1 Khái niệm

A* là một giải thuật tìm kiếm theo đường đi tốt nhất (best-first search) khá phổ biến. Mục tiêu của giải thuật này là tìm đường đi ngắn nhất hoặc có chi phí thấp nhất từ điểm bắt đầu đến điểm kết thúc.

Giải thuật A* hoạt động dựa trên hai yếu tố:

- Chi phí thực tế để đi từ điểm bắt đầu đến điểm hiện tại
- Chi phí ước tính (heuristic) để đi từ trạng thái hiện tại đến điểm đích

Công thức: $f(n) = h(n) + g(n)$, trong đó:

- n là một trạng thái (state) hoặc nút (node) (Trong game Sokoban, n là vị trí cầu người chơi và vị trí của tất cả các thùng)
- $g(n)$ là chi phí thực tế đã mất để đi từ trạng thái bắt đầu đến trạng thái n .
- $h(n)$ là hàm định giá để đi từ trạng thái n đến trạng thái đích.
- $f(n)$ là tổng chi phí ước tính của trạng thái n .

Mô tả chi tiết thuật toán A*:

- Khởi tạo: Chọn trạng thái xuất phát. Thêm nó vào hàng đợi ưu tiên.
- Duyệt: Lặp lại cho đến khi hàng đợi ưu tiên hết phần tử
 - Lấy trạng thái n có $f(n)$ nhỏ nhất ra khỏi hàng đợi
 - Nếu n là trạng thái đích, thuật toán kết thúc và trả về đường đi
 - Đánh dấu n là đã duyệt qua
 - Với mỗi trạng thái m kề với n (một bước di chuyển/ đẩy hộp), nếu m chưa được duyệt, tính $f(m)$ và thêm m vào hàng đợi ưu tiên.
- Kết thúc: Thuật toán dừng lại khi tìm thấy đích hoặc hàng đợi ưu tiên rỗng (không tìm thấy đường đi).

3.2 Chiến lược tìm kiếm

A* là một thuật toán tìm kiếm có trọng số, kết hợp giữa chi phí thực tế đã đi được và chi phí ước lượng còn lại đến mục tiêu thông qua hàm heuristic. Trong Sokoban, A* được sử dụng để tìm chuỗi hành động tối ưu giúp người chơi di chuyển và đẩy các hộp đến vị trí đích với tổng số bước ít nhất. Khác với DFS, A* không duyệt mù quáng mà luôn ưu tiên mở rộng các trạng thái có giá trị đánh giá $f(n) = g(n) + h(n)$ nhỏ nhất, trong đó $g(n)$ là chi phí thực tế từ trạng thái ban đầu và $h(n)$ là ước lượng chi phí còn lại đến mục tiêu.

Khi áp dụng A* cho Sokoban, thuật toán bắt đầu từ trạng thái ban đầu và liên tục mở rộng các trạng thái có triển vọng nhất dựa trên giá trị $f(n)$. Mỗi trạng thái được lưu vào hàng đợi ưu tiên (priority queue) để đảm bảo trạng thái tốt nhất luôn được xét trước. Quá trình tìm kiếm kết thúc khi trạng thái mục tiêu — tức tất cả các hộp đã ở đúng vị trí đích — được tìm thấy. Nhờ sử dụng thông tin heuristic, A* có khả năng định hướng tìm kiếm, tránh duyệt các nhánh không cần thiết và thường đạt được lời giải tối ưu trong thời gian ngắn hơn so với các thuật toán duyệt mù như DFS.

4 Hệ thống trò chơi

4.1 Các thư viện được áp dụng

Dự án Sokoban of Magic được phát triển bằng ngôn ngữ Python, cụ thể là phiên bản Python 3.12. Việc lựa chọn Python mang lại lợi thế về tốc độ phát triển, cộng đồng hỗ trợ lớn và khả năng tích hợp dễ dàng các thư viện hỗ trợ phát triển game (như Pygame và NumPy).

Dưới đây là danh sách các thư viện chính được sử dụng trong dự án:

- **os**: Cung cấp các chức năng tương tác với hệ điều hành, chủ yếu được dùng để quản lý đường dẫn file (`os.path.join`, `os.getcwd`) cho các file map và assets.
- **time**: Dùng để đo lường thời gian thực thi của thuật toán (`time.time()`) và tính toán giới hạn thời gian (`Timeout`).
- **psutil**: Được sử dụng để đo lường hiệu suất hệ thống, cụ thể là đo lường và giám sát lượng RAM mà tiến trình Python chiếm dụng trong suốt quá trình chạy thuật toán.
- **threading**: Quản lý việc chạy đồng thời luồng chính (Pygame GUI) và luồng AI (`run_ai`), ngăn chặn giao diện bị đóng băng trong khi thuật toán đang tính toán.
- **numpy**: Thư viện cơ bản cho tính toán, được sử dụng để xử lý dữ liệu map dưới dạng mảng (array) đa chiều, giúp thao tác vị trí (position) và ma trận trạng thái game hiệu quả.
- **pygame**: Cung cấp giao diện đồ họa cho trò chơi (GUI), xử lý vòng lặp game, sự kiện người dùng, và hiển thị map, nhân vật, và kết quả.

4.2 Áp dụng thuật toán

4.2.1 DFS

Cách thức thực hiện DFS:

DFS (Depth First Search) là một thuật toán tìm kiếm trong đồ thị hoặc cây, hoạt động dựa trên việc duyệt sâu nhất có thể trong không gian trạng thái trước khi quay lại để duyệt các nhánh khác. Trong trò chơi Sokoban, DFS được sử dụng để tìm một chuỗi hành động giúp người chơi di chuyển các hộp từ vị trí ban đầu đến các vị trí mục tiêu. Thuật toán sử dụng ngăn xếp (stack) để lưu trữ các trạng thái cần xét, đảm bảo trạng thái được đi sâu nhất sẽ được mở rộng trước.

- **Bước 1**: Khởi tạo ngăn xếp (stack) với phần tử đầu tiên là trạng thái ban đầu, gồm vị trí người chơi, vị trí các hộp và chuỗi hành động rỗng. Đồng thời khởi tạo tập `exploredSet` để lưu các trạng thái đã được duyệt, tránh lặp lại. Nếu trạng thái ban đầu đã là trạng thái đích (tất cả hộp ở đúng vị trí mục tiêu), thuật toán kết thúc ngay.
- **Bước 2**: Lặp cho đến khi stack rỗng hoặc tìm thấy lời giải:

- Lấy phần tử trên đỉnh stack ra để xét (theo nguyên tắc LIFO).
 - Nếu trạng thái hiện tại đã được duyệt thì bỏ qua, ngược lại thêm vào explored-Set.
 - Kiểm tra xem các hộp đã ở đúng vị trí mục tiêu (isEndState) hay chưa. Nếu đúng, thuật toán in ra chuỗi hành động và kết thúc.
 - Nếu chưa đạt mục tiêu, sinh ra các trạng thái con hợp lệ bằng hàm legalActions(). Với mỗi hành động hợp lệ, cập nhật trạng thái mới của người chơi và hộp bằng updateState().
 - Nếu trạng thái mới không hợp lệ (ví dụ hộp bị kẹt, xác định bởi isFailed()), bỏ qua. Nếu hợp lệ, thêm vào stack cùng chuỗi hành động mới (actions + action[-1]) để tiếp tục duyệt sâu.
- Bước 3: Nếu stack rỗng và chưa tìm thấy trạng thái đích, thuật toán kết thúc và trả về kết quả không tìm thấy.

Hàm dfs_search() đóng vai trò bao bọc quá trình tìm kiếm. Nó khởi tạo dữ liệu trò chơi (gameState, posWalls, posGoals) từ bản đồ Sokoban nhập vào, sau đó gọi hàm dfsSearch() để thực hiện tìm kiếm. Hàm này cũng kiểm tra thời gian chạy, nếu vượt quá giới hạn (time_limit = 1800 giây), chương trình dừng và trả về 'x'.

Cuối cùng, hàm test_dfs() được dùng để đọc bản đồ từ file đầu vào (input/1.txt), gọi dfs_search() và in ra chuỗi hành động kết quả. Nhờ đó, ta có thể kiểm tra và đánh giá khả năng hoạt động của thuật toán trên các bài toán Sokoban khác nhau.

Kiểm tra tính khả thi với DFS:

- Trước khi mở rộng một nhánh trong quá trình tìm kiếm, thuật toán DFS sẽ kiểm tra tính hợp lệ của trạng thái mới được sinh ra từ bước di chuyển.
- Bằng cách tạo trạng thái mới và đánh giá xem trạng thái đó có hợp lệ hay không (ví dụ: người chơi không đi xuyên tường, hộp không bị đẩy vào góc chết hoặc vị trí

Tạo các cấp độ mới với DFS:

- DFS có thể được áp dụng để sinh ra các cấp độ mới trong trò chơi Sokoban.
- Bằng cách sử dụng DFS từ trạng thái ban đầu, ta có thể mở rộng theo từng nhánh di chuyển, tạo ra các trạng thái mới và ghi nhận các trạng thái đã thăm để tránh lặp lại.
- Cách tiếp cận này giúp tạo ra những cấp độ mới với độ phức tạp cao hơn, do DFS có xu hướng đi sâu theo một nhánh trước, từ đó hình thành các tình huống khó khăn hơn cho người chơi.

4.2.2 A*

A* là một thuật toán tìm kiếm đồ thị dựa trên việc khám phá các trạng thái tốt nhất dựa trên hàm chi phí $f(n) = g(n) + h(n)$.

- Thuật toán bắt đầu từ một trạng thái gốc (bản đồ ban đầu).
- A* sử dụng cơ chế hàng đợi ưu tiên (Priority Queue) để lưu trữ các trạng thái chờ xét, đảm bảo rằng trạng thái có tổng chi phí $f(n)$ thấp nhất sẽ được duyệt trước.

Cách thực hiện A*:

- **Bước 1:** Khởi tạo hàng đợi ưu tiên (Priority Queue) và đặt trạng thái ban đầu vào đó với chi phí $g = 0$ và $f = h(\text{trạng thái ban đầu})$. Khởi tạo một tập hợp `exploredSet` để lưu các trạng thái đã duyệt.
- **Bước 2:** Lặp lại cho đến khi hàng đợi ưu tiên rỗng hoặc tìm thấy lời giải:
 - Lấy trạng thái hiện tại có $f(n)$ thấp nhất ra khỏi hàng đợi ưu tiên.
 - Kiểm tra xem trạng thái hiện tại có phải trạng thái đích (tất cả hộp đã được đặt vào đúng vị trí) hay không. Nếu đúng, thuật toán kết thúc và trả về kết quả (chuỗi hành động).
 - Nếu trạng thái này đã có trong `exploredSet`, bỏ qua.
 - Thêm trạng thái hiện tại vào `exploredSet`.
 - Sinh ra tất cả các trạng thái con có thể từ trạng thái hiện tại bằng cách xét các nước đi hợp lệ của người chơi (đi hoặc đẩy thùng).

Với mỗi trạng thái con:

- Bỏ qua nếu trạng thái đã xuất hiện trong `exploredSet`.
- Bỏ qua nếu trạng thái không khả thi (ví dụ: hộp bị kẹt vào góc chết, được kiểm tra bởi hàm `isFailed`).
- Nếu hợp lệ: Tính chi phí $g(\text{con}) = g(\text{hiện tại}) + \text{chi phí hành động}$.
- Tính chi phí ước tính $h(\text{con})$.
- Tính tổng chi phí $f(\text{con}) = g(\text{con}) + h(\text{con})$.
- Thêm trạng thái con vào hàng đợi ưu tiên với độ ưu tiên chính là $f(\text{con})$.
- **Bước 3:** Nếu hàng đợi ưu tiên rỗng và chưa tìm thấy trạng thái đích, thuật toán kết thúc và trả về kết quả không tìm thấy.

Kiểm tra tính khả thi:

- Giống như DFS, trước khi mở rộng một nhánh, thuật toán A* sẽ kiểm tra tính hợp lệ của trạng thái mới (người chơi không đi xuyên tường...)
- Quan trọng hơn, việc kiểm tra "thế chết" deadlock (hộp bị đẩy vào góc) là cực kỳ quan trọng. Trong A*, việc này giúp loại bỏ các nhánh tìm kiếm vô ích, giúp thuật toán không lãng phí thời gian vào các trạng thái không thể thắng.

Chức năng của Hàm g trong giải thuật của nhóm:

Hàm g tính tổng số bước người chơi tự di chuyển (không đẩy thùng). Thuật toán A* sẽ cố gắng tìm đường đi nào có tổng số bước di chuyển này là thấp nhất.

Chức năng của Hàm heuristic (Hàm h) của nhóm:

Hàm h là tổng khoảng cách (Manhattan) từ các thùng chưa hoàn thành đến các đích còn trống. Giá trị này càng nhỏ thì thuật toán A* càng chắc chắn rằng trạng thái này gần với chiến thắng.

Hàm Manhattan:

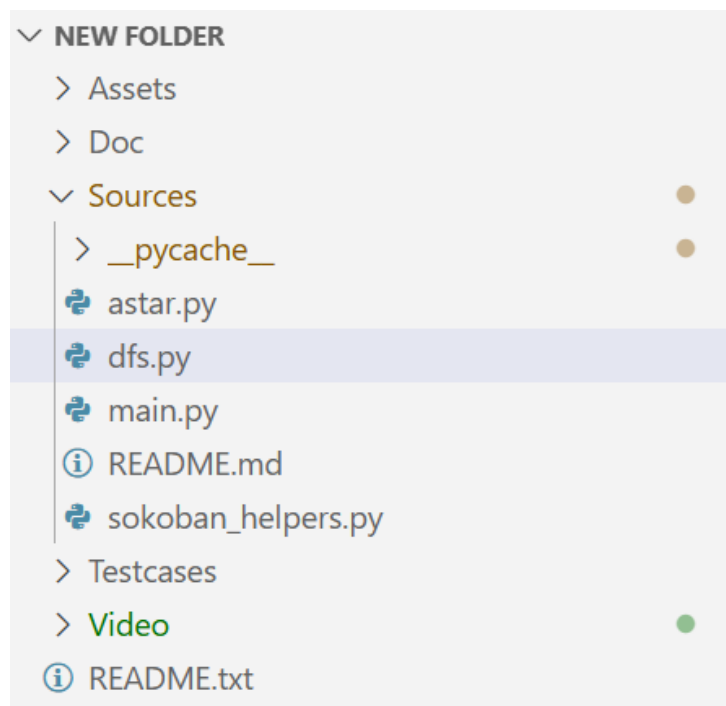
Hàm Manhattan tính khoảng cách Manhattan (tổng giá trị tuyệt đối) giữa hai điểm trên mặt phẳng. Công thức tính khoảng cách Manhattan giữa hai điểm $(x1, y1)$ và $(x2, y2)$ là $h = |x2 - x1| + |y2 - y1|$.

Trong trò chơi Sokoban, vị trí hiện tại của người chơi là điểm bắt đầu và vị trí của hộp cần di chuyển đến là điểm mục tiêu.

Hàm Manhattan ước lượng chi phí còn lại từ vị trí hiện tại đến vị trí mục tiêu dựa trên khoảng cách Manhattan, giúp thuật toán A* tìm kiếm đường đi tối ưu

4.3 Trình bày về thư mục code và cách chạy trò chơi

4.3.1 Trình bày thư mục code



Hình 4.1: Cấu trúc thư mục

Cấu trúc thư mục được sử dụng bao gồm:

- **Assets:** Thư mục chứa các ảnh giao diện trò chơi cần thiết cho đề tài. Đây là nơi lưu trữ các file ảnh của nhân vật, background,...
- **Doc:** Thư mục chứa file báo cáo định dạng pdf
- **Sources:** Đây là thư mục chứa toàn bộ mã nguồn. Chứa tất cả các module Python chính để chạy game và hiện thực thuật toán AI.
- **Testcases:** Thư mục chứa thông tin định dạng các map ở dạng text. Cụ thể là các file định dạng map Sokoban (1.txt, 2.txt,...) để thuật toán có thể tải và giải quyết.
- **Video:** Thư mục chứa bài thuyết trình và link video demo

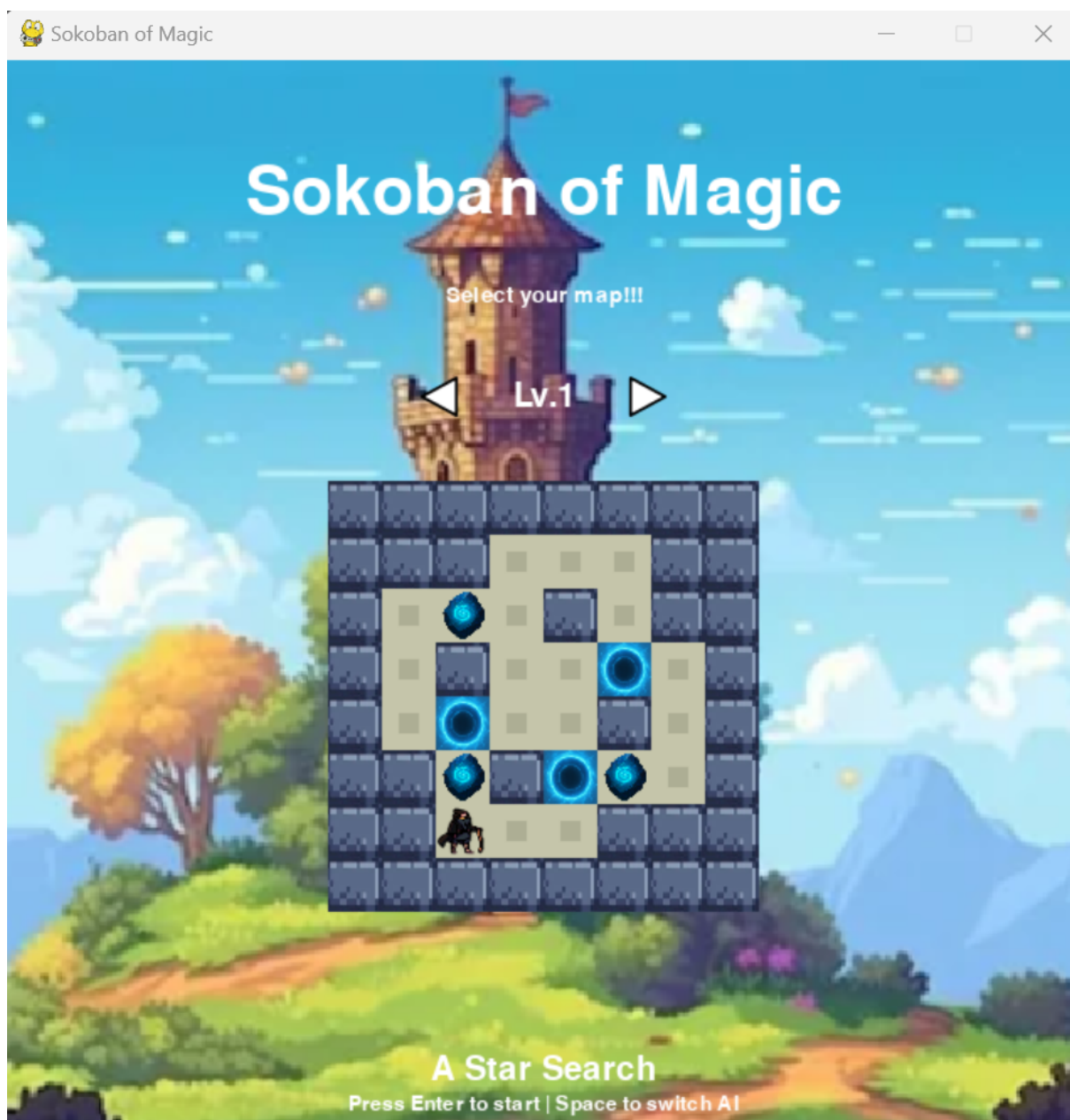
Để xây dựng giao diện game và các thuật toán, nhóm đã tạo các file python để hiện thực và để trong thư mục Sources:

- **astar.py:** Chứa logic triển khai thuật toán tìm kiếm A* Search (có Heuristic) để giải quyết Sokoban.
- **dfs.py:** Chứa logic triển khai thuật toán tìm kiếm Depth-First Search để giải quyết Sokoban.
- **sokoban_helpers.py:** Chứa các hàm logic nền tảng quan trọng mà cả A* và DFS đều cần: chuyển đổi định dạng map, kiểm tra hành động hợp lệ (legalActions), cập nhật trạng thái (updateState), và các hàm kiểm tra Deadlock (isFailed)

- **main.py**: Xử lý giao diện người dùng Pygame, vòng lặp game, , xử lý input, đo lường hiệu suất (RAM, Time), và chạy các thuật toán AI trong luồng (threading).

4.3.2 Hướng dẫn chạy trò chơi

Sau khi tải và giải nén mã nguồn thì ta sẽ có cấu trúc thư mục giống như đã trình bày ở trên. Đầu tiên chuyển sang thư mục nguồn là Sources rồi dùng lệnh "python main.py" để chạy trò chơi. Ta sẽ được giao diện khởi tạo:

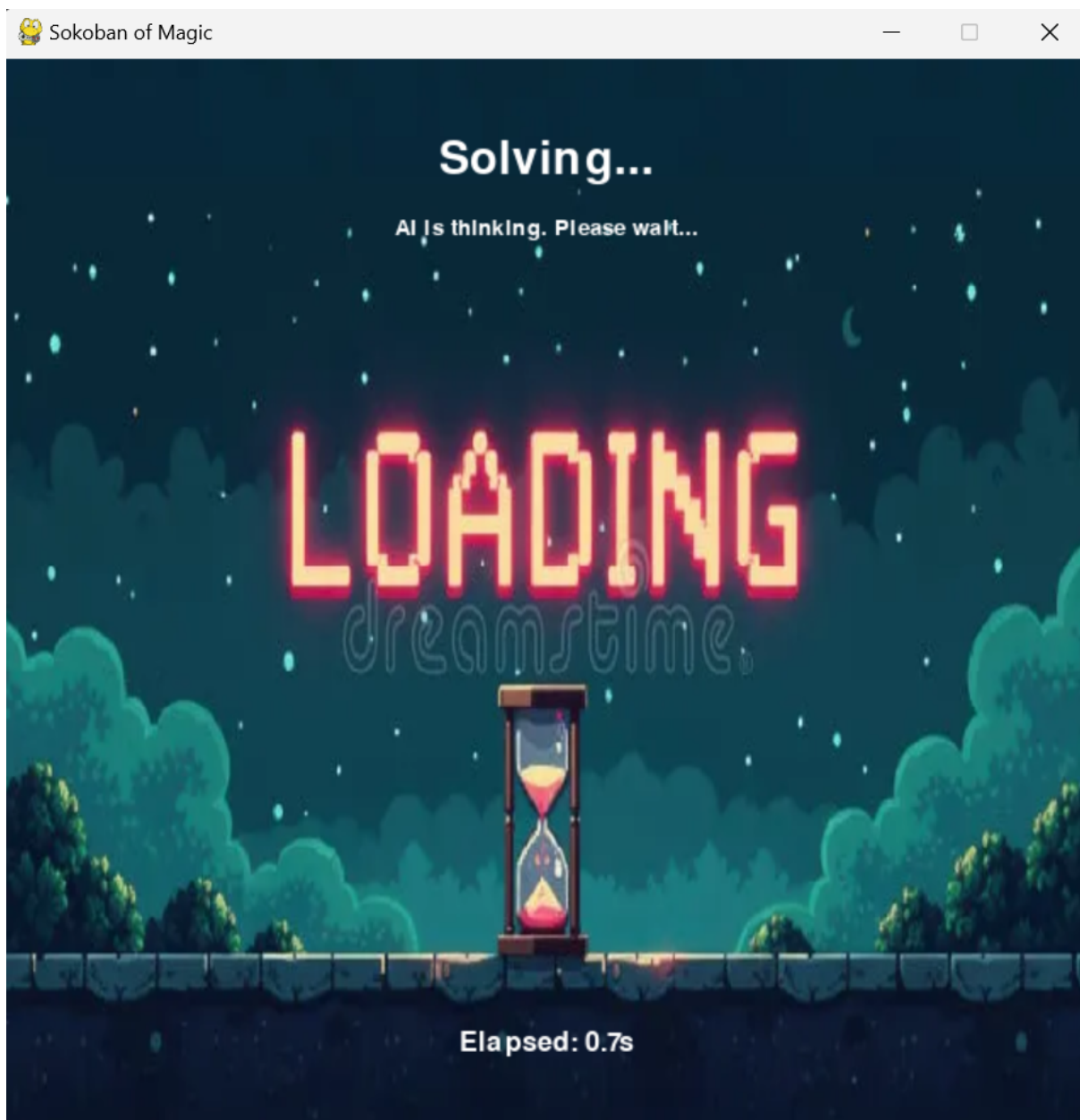


Hình 4.2: Giao diện khởi đầu trò chơi

Giao diện sẽ hiển thị màn chơi ở từng cấp độ, có thể dùng phím mũi tên để chuyển sang màn khác. Đồng thời, có thể chọn thuật toán để giải màn chơi. Ở đây ta có thể chọn 2 thuật toán đã hiện thực là A* và DFS bằng cách nhấn phím Space.

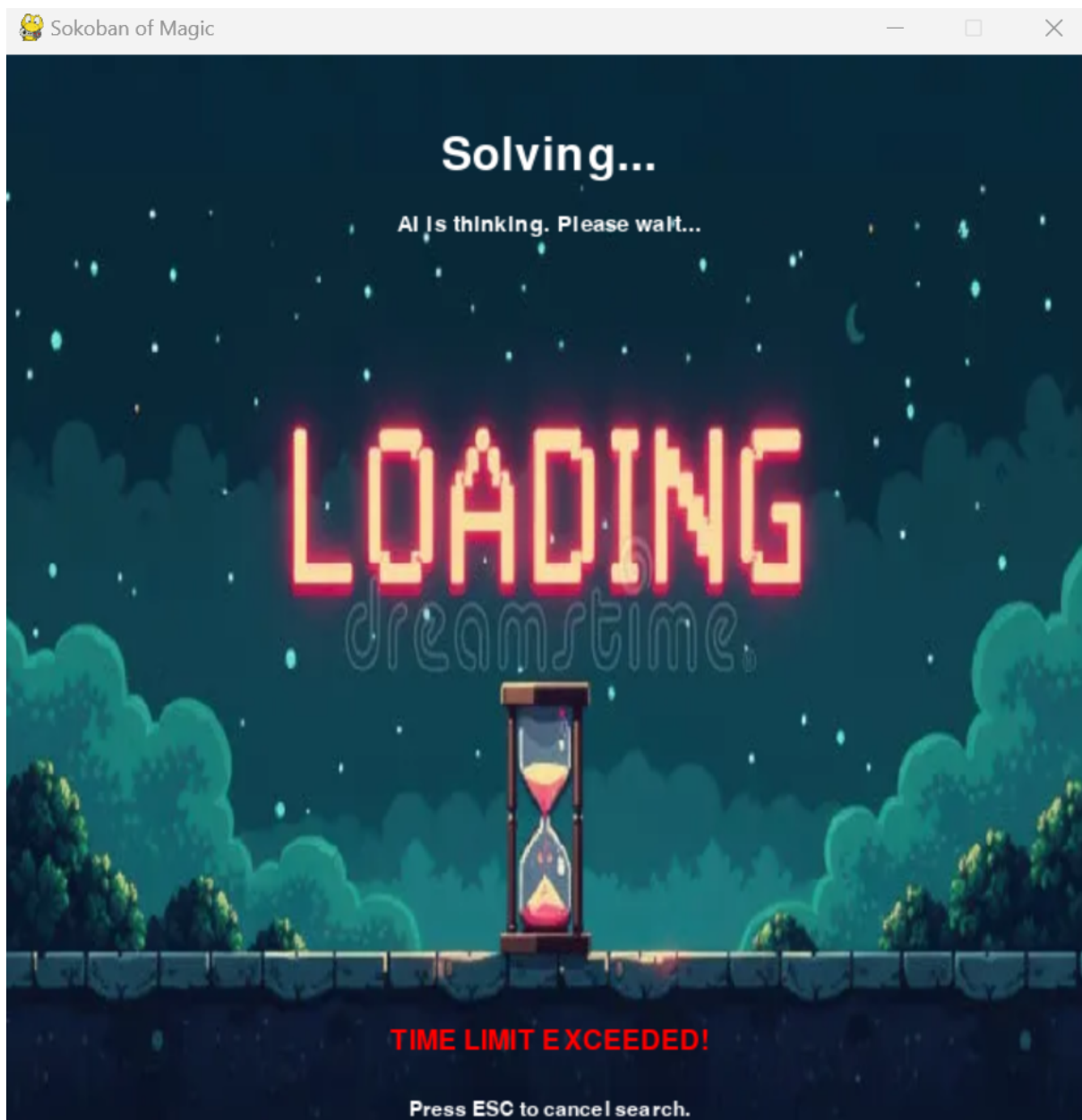
Sau khi đã xác nhận màn chơi và thuật toán sử dụng, nhấn Enter để bắt đầu màn

chơi. Giao diện sẽ hiển thị một màn hình chờ trong khi đợi thuật toán chạy xong, có bộ đếm để tính thời gian thuật toán chạy.



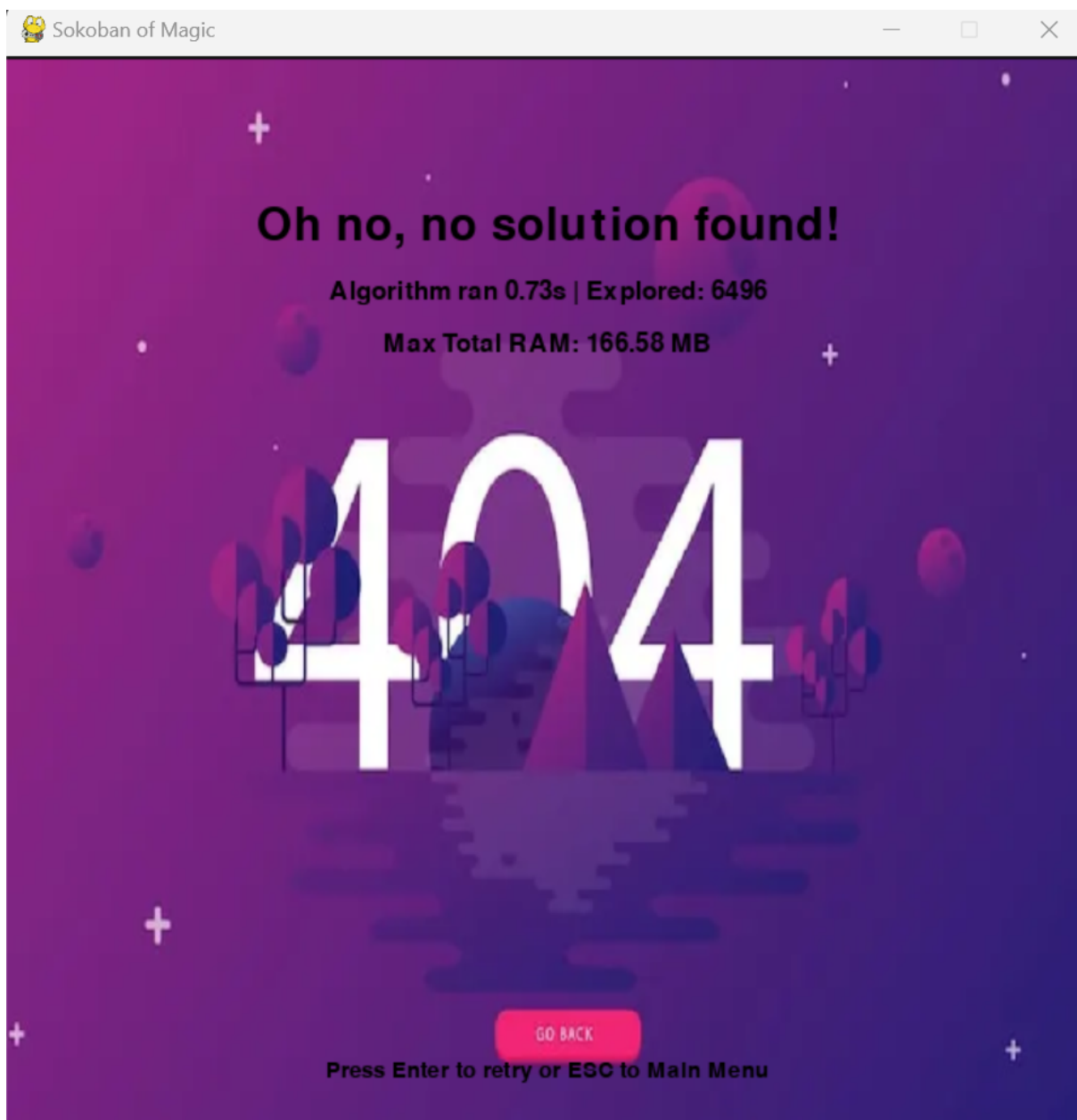
Hình 4.3: *Giao diện chờ trong kì thuật toán chạy*

Trong trường hợp thời gian chạy thuật toán vượt quá 20 phút thì giao diện sẽ hiển thị cảnh báo "TIME LIMIT EXCEEDED" ở dưới. Có thể chờ thuật toán chạy xong nếu để nguyên do tiến trình vẫn chạy, sau khi thuật toán chạy xong thì nhấn ESC và sẽ hiện kết quả, tuy nhiên không khuyến khích cách này vì không biết chính xác thuật toán cần bao lâu. Nếu muốn chọn lại màn chơi thì cần tắt trò chơi này và chạy lại theo hướng dẫn ở trên.



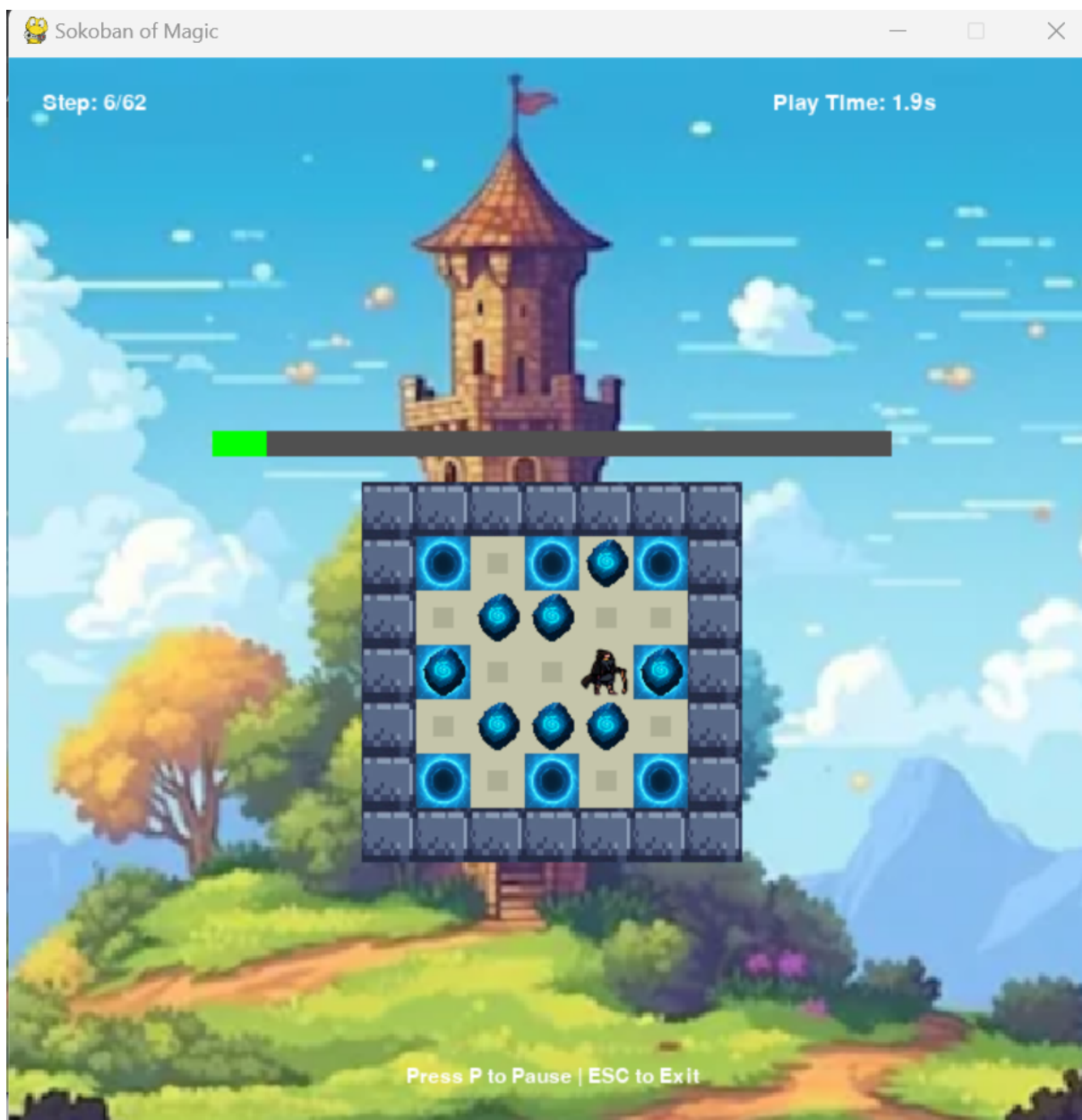
Hình 4.4: *Giao diện hiển thị thời gian chạy thuật toán vượt quá giới hạn(20 phút)*

Trong trường hợp thuật toán đã chạy xong mà không tìm thấy lời giải, màn hình sẽ hiển thị thông báo không tìm thấy. Ta có thể nhấn nút ESC để quay lại giao diện ban đầu hoặc nút Enter để chạy lại thuật toán nếu cho rằng có lỗi xảy ra.



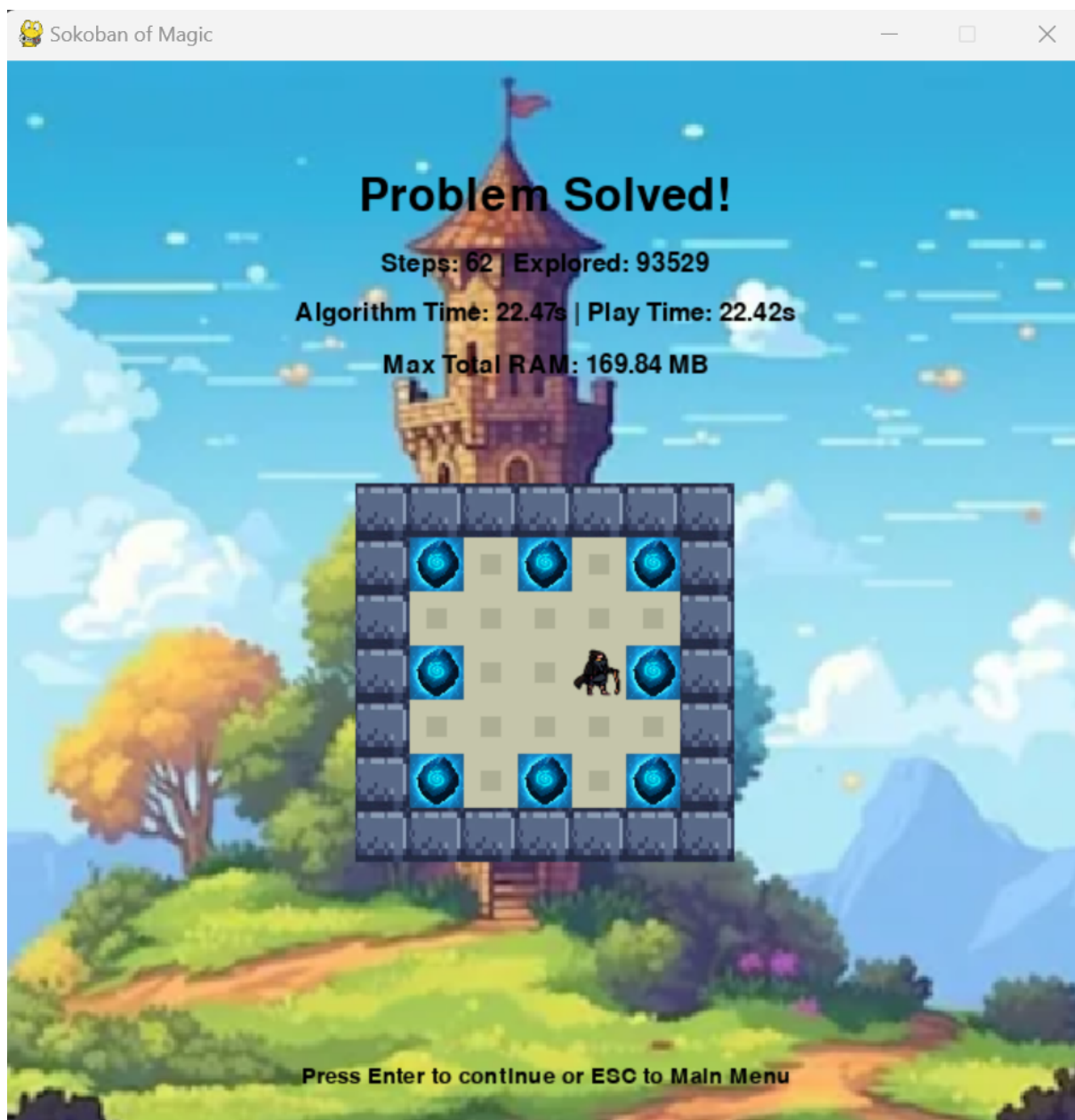
Hình 4.5: *Giao diện thông báo không tìm thấy lời giải*

Trong trường hợp thuật toán tìm thấy lời giải, giao diện sẽ chuyển sang màn chơi và hệ thống sẽ tự động giải quyết bài toán dựa theo đường đi thuật toán chọn. Trên giao diện có hiển thị số bước di chuyển cần thiết để giải ở góc trên bên trái, thời gian chơi ở góc trên bên phải, và 1 thanh tiến trình ở giữa. Nếu muốn tạm dừng màn chơi thì nhấn P, nếu muốn thoát ra chọn màn chơi khác thì nhấn ESC.



Hình 4.6: *Giao diện chạy màn chơi*

Sau khi hệ thống đã giải quyết xong màn chơi, giao diện sẽ hiển thị một số thông số để đánh giá. Các thông số bao gồm thời gian thuật toán chạy, thời gian của màn chơi, số bước để giải quyết bài toán, số trạng thái đã duyệt trong thuật toán, bộ nhớ cần dùng trong khi chạy



Hình 4.7: Giao diện hiển thị kết quả

5 Phát triển và đánh giá

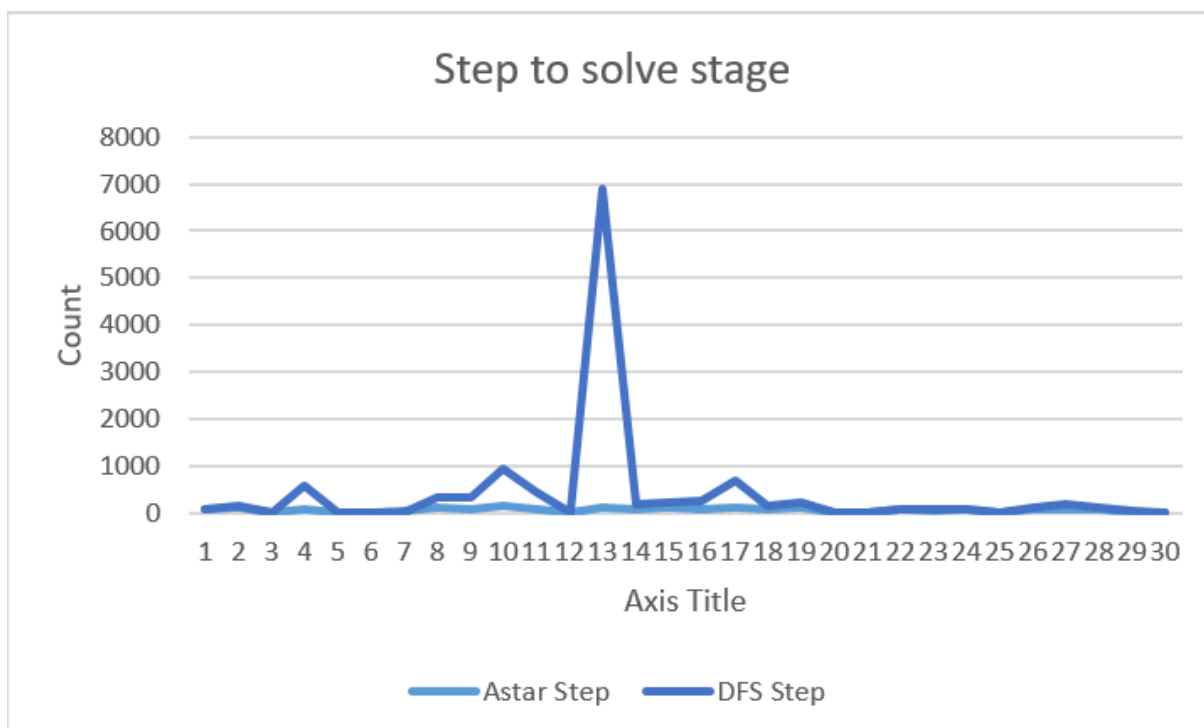
5.1 Kết quả

Stage	Astar				DFS			
	Step	State	RAM(MB)	Time(s)	Step	State	RAM(MB)	Time(s)
1	96	1093	61.74	0.21	82	538	62.36	0.1
2	119	12505	68.58	2.17	143	5486	68.63	0.81
3	NULL	133206	205.13	41.94	NULL	133206	205.38	30.46
4	72	98200	132.59	21.94	576	545603	356.62	116.93
5	OT	OT	OT	OT	OT	OT	OT	OT
6	OT	OT	OT	OT	OT	OT	OT	OT
7	62	93529	259.43	22.42	OT	OT	OT	OT
8	106	2513	187.84	0.42	324	3744	182.75	0.51
9	89	4149	182.74	0.62	327	1345	182.75	0.2
10	157	17554	172.19	3.1	957	37453	160.23	5.77
11	87	8776	151.06	1.23	429	1883	142.1	0.2
12	NULL	2388	123.72	0.41	NULL	2388	125.33	0.41
13	138	118609	139	25.62	6898	103517	123.39	16.98
14	83	3328	117.78	0.51	191	2095	115.8	0.31
15	125	9983	116.34	1.77	243	2859	116.34	0.51
16	83	3800	111.17	0.62	257	1816	111.19	0.21
17	136	22295	111.18	4.27	680	25006	110.77	3.88
18	72	3619	107.64	0.52	144	1189	107.69	0.2
19	114	9218	107.35	1.63	238	1970	106.56	0.31
20	NULL	6496	107.2	1.02	NULL	6496	107.2	0.81
21	NULL	52489	110.16	11.41	NULL	52489	108.84	9.43
22	75	756	103.84	0.1	85	734	103.83	0.1
23	51	1106	102.84	0.21	98	1749	100.83	0.2
24	92	959	99.82	0.21	92	654	97.8	0.11
25	NULL	5239	96.84	1.64	NULL	5239	94.83	1.47
26	103	17407	94.87	3.32	115	1612	93.8	0.31
27	79	9676	94.46	1.87	185	281485	213.62	54.27
28	73	8616	205.68	1.45	135	193.74	193.74	0.63
29	33	194	188.71	0.1	35	1366	186.75	0.21
30	NULL	30823	181.69	6.21	NULL	30823	141.96	5.71

Hình 5.1: Bảng tổng hợp kết quả chạy trên 30 map

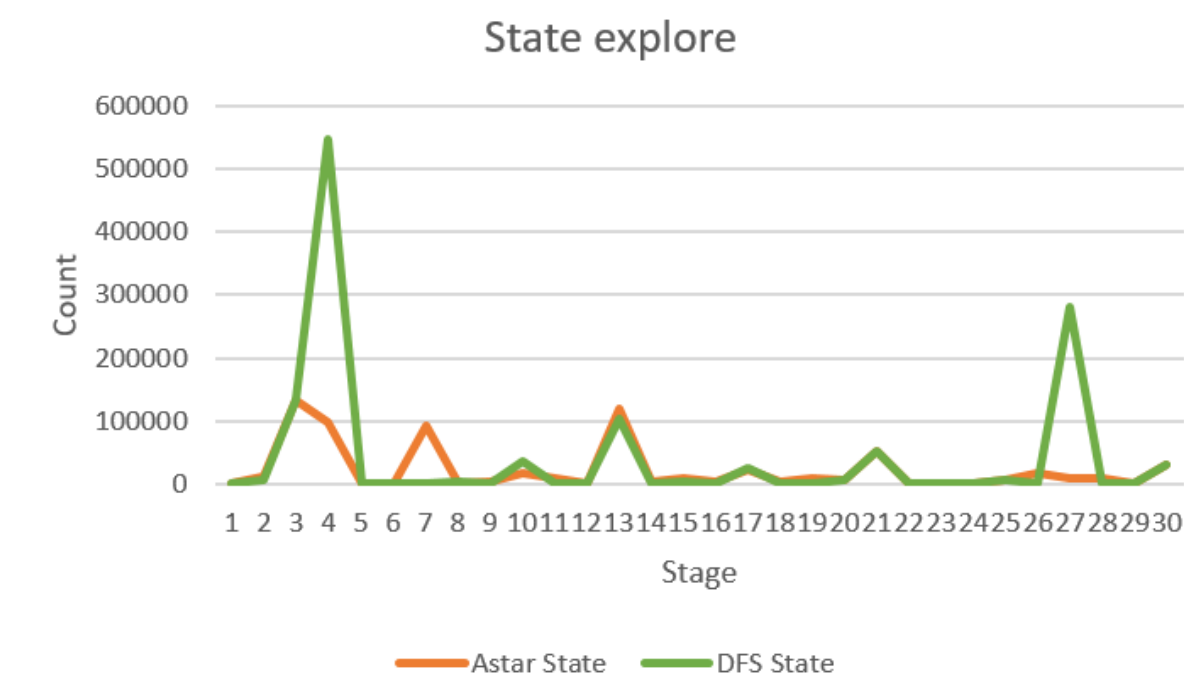
Với các màn chơi có step có giá trị NULL có nghĩa là không có lời giải. Các màn chơi có bộ đánh giá là OT nghĩa là đã quá thời gian (Over time) giới hạn. Trong bài toán này giới hạn là 20 phút.

Dưới đây là các biểu đồ so sánh chi tiết giữa 2 thuật toán



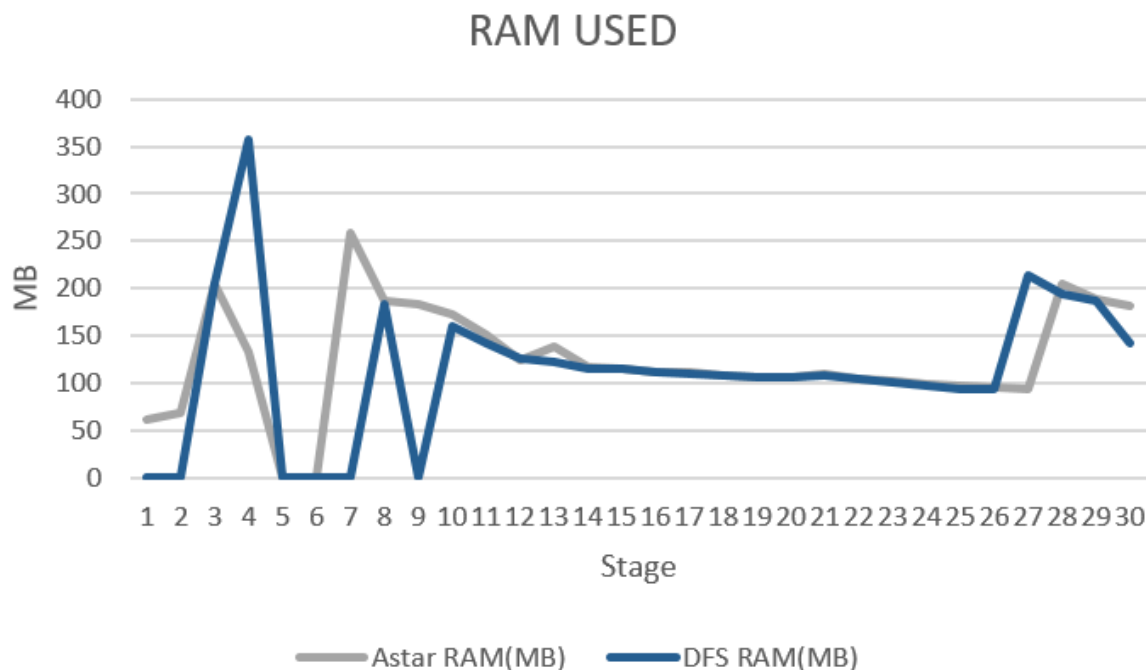
Hình 5.2: Biểu đồ so sánh về số bước cần đi để giải bài toán trên 30 map

Dựa trên biểu đồ đánh giá 30 màn chơi, thuật toán A* Search thể hiện sự vượt trội rõ ràng về chất lượng lời giải, trong khi DFS Search cho thấy tính không ổn định và thiếu hiệu quả. A* đảm bảo tính tối ưu bằng cách duy trì số bước giải quyết thấp và nhất quán trên tất cả các màn chơi, xác nhận khả năng tìm ra đường đi ngắn nhất. Ngược lại, DFS không quan tâm đến chi phí, dẫn đến sự biến động lớn về số bước.



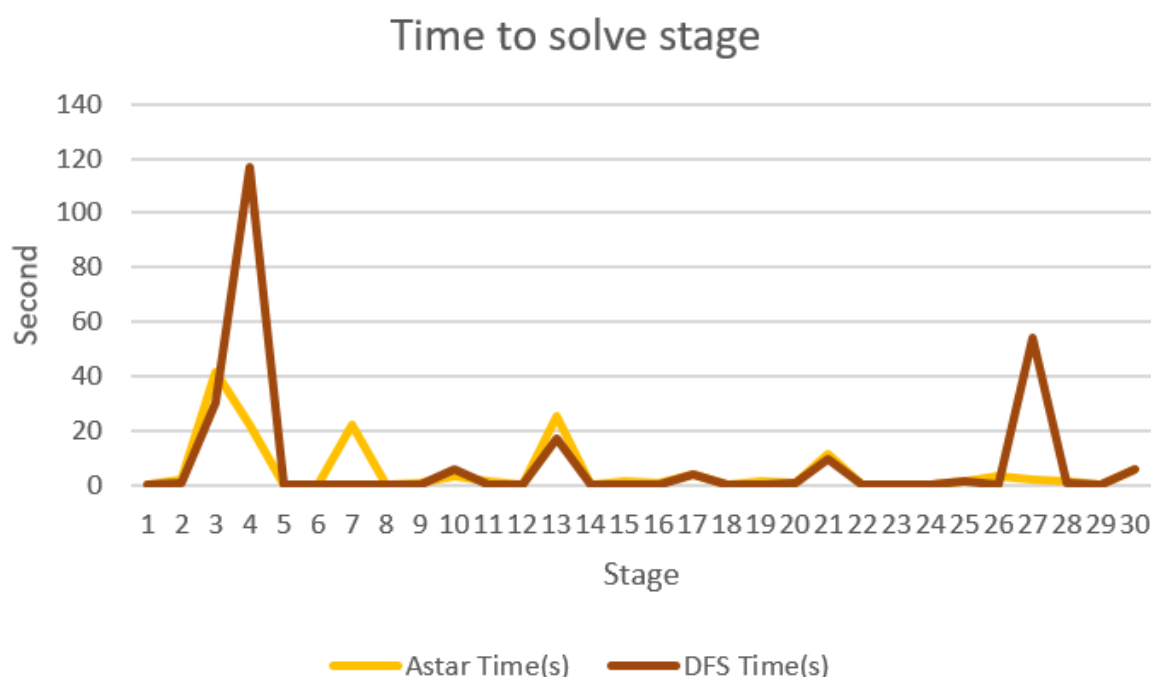
Hình 5.3: Biểu đồ so sánh về số trạng thái đã duyệt trên 30 map

Trong khi DFS Search khám phá không hiệu quả, dẫn đến sự bùng nổ tìm kiếm với số lượng trạng thái lên đến hơn 500,000 (ví dụ Lv. 4), thì A* Search luôn duy trì mức khám phá ổn định và thấp hơn đáng kể.



Hình 5.4: Biểu đồ so sánh về số lượng bộ nhớ đã dùng khi chạy thuật toán trên 30 map

Biểu đồ cho thấy A* (đường màu xám) thường có nhu cầu bộ nhớ cao hơn, dao động khoảng 100-200 MB, do phải duy trì một Priority Queue lớn để đảm bảo tính tối ưu. Ngược lại, DFS Search (đường màu xanh đậm) có xu hướng sử dụng ít bộ nhớ hơn (do dùng Stack), nhưng lại thể hiện sự tăng vọt đột biến về tiêu thụ RAM (đạt đỉnh 356 MB tại Lv. 4) khi DFS đi sâu vào các nhánh vô ích và phải lưu trữ một lượng lớn trạng thái. Do đó DFS có rủi ro về bộ nhớ khi gặp phải các bài toán có độ sâu lớn, trong khi A* thể hiện sự cân đối



Hình 5.5: Biểu đồ so sánh về thời gian để giải 30 map

Ta có thể thấy A* là thuật toán vượt trội về tốc độ thực thi tổng thể. Nhờ có Heuristic giúp giảm thiểu đáng kể số lượng trạng thái cần khám phá, A* có thể giải quyết hầu hết các màn chơi một cách nhanh chóng và ổn định. Đối với DFS, mặc dù có thể nhanh trong các màn chơi đơn giản, nhưng luôn tiềm ẩn rủi ro chạy quá lâu, vượt quá giới hạn thời gian khi đối diện với các màn chơi phức tạp có nhiều nhánh tìm kiếm sâu.

5.2 Đối với DFS:

Độ phức tạp thời gian và không gian:

DFS có độ phức tạp thời gian cao, đặc biệt trong các bản đồ Sokoban lớn với nhiều hộp và chướng ngại vật. Vì DFS có xu hướng đi sâu vào một nhánh cho đến khi không thể mở rộng, số lượng trạng thái phải duyệt có thể rất lớn, dẫn đến thời gian tìm kiếm lâu. Tuy nhiên, DFS sử dụng ít bộ nhớ hơn A*, vì nó chỉ lưu trữ các trạng thái trên nhánh hiện tại thay vì toàn bộ cây trạng thái đang mở như A*. Điều này giúp DFS có lợi thế về mặt bộ nhớ nhưng kém hơn về tốc độ tìm kiếm và hiệu quả tổng thể.

Không đảm bảo tối ưu hóa:

DFS không đảm bảo tìm được đường đi ngắn nhất. Khi áp dụng vào Sokoban, DFS có thể tìm thấy lời giải đầu tiên nhưng đó chưa chắc là lời giải tối ưu. Ngược lại, A* sử dụng hàm heuristic để đánh giá và ưu tiên các trạng thái có triển vọng tốt nhất, giúp tìm được lời giải ngắn hơn và hiệu quả hơn. Vì vậy, A* thường tối ưu hơn DFS về chất lượng kết quả.

Khả năng rơi vào vòng lặp hoặc nhánh vô hạn:

DFS có thể rơi vào vòng lặp nếu không kiểm soát trạng thái đã duyệt, ví dụ khi người chơi di chuyển qua lại giữa hai vị trí. Điều này khiến thuật toán mất nhiều thời gian hoặc không bao giờ dừng. Trong khi đó, A* luôn kiểm tra và đánh giá lại trạng thái, nhờ vậy hạn chế được việc duyệt lặp và tránh các nhánh không cần thiết.

Không sử dụng thông tin heuristic:

DFS không sử dụng thông tin heuristic, nghĩa là không có cơ chế đánh giá xem trạng thái nào gần với mục tiêu hơn. Vì vậy, DFS duyệt mù quảng theo thứ tự hành động, trong khi A* sử dụng các heuristic như khoảng cách Manhattan giữa hộp và ô đích, giúp quá trình tìm kiếm nhanh và có định hướng hơn.

5.2.1 Đối với A*:

Độ phức tạp thời gian và không gian:

A* có độ phức tạp thời gian cao do phải mở rộng và đánh giá nhiều trạng thái, đặc biệt khi không gian trạng thái lớn như trong trò chơi Sokoban với nhiều hộp và chướng ngại vật. Tuy nhiên, nhờ sử dụng hàm heuristic để ước lượng chi phí đến mục tiêu, A* thường duyệt ít trạng thái hơn so với các thuật toán tìm kiếm mù như DFS, giúp rút ngắn đáng kể thời gian tìm kiếm. Mặt khác, A* yêu cầu bộ nhớ lớn để lưu trữ toàn bộ các trạng thái đã mở (Open list) và đã duyệt (Closed list), khiến chi phí không gian của nó cao hơn DFS.

Đảm bảo tối ưu hóa: Khác với DFS, A* có khả năng đảm bảo tìm được đường đi ngắn nhất nếu hàm heuristic được thiết kế hợp lý (admissible). Khi áp dụng vào Sokoban, A* không chỉ tìm thấy lời giải mà còn tìm ra lời giải có số bước di chuyển tối thiểu, giúp đạt hiệu quả tối ưu cả về chất lượng và thời gian giải quyết.

Tránh lặp và cải thiện hiệu quả: A* có cơ chế kiểm soát và đánh giá lại trạng thái thông qua các tập Open và Closed, nhờ đó hạn chế việc duyệt lại các trạng thái cũ và tránh rơi vào vòng lặp như DFS. Ngoài ra, việc sử dụng giá trị $f(n) = g(n) + h(n)$ cho phép A* tập trung vào các trạng thái có triển vọng nhất, giúp quá trình tìm kiếm có định hướng rõ ràng hơn và loại bỏ nhiều nhánh không cần thiết.

Sử dụng thông tin heuristic: Điểm mạnh lớn nhất của A* nằm ở việc khai thác thông tin heuristic để dẫn đường cho quá trình tìm kiếm. Trong bài toán Sokoban, các hàm heuristic như khoảng cách Manhattan giữa các hộp và vị trí đích giúp A* ước lượng được hướng đi ngắn nhất và hiệu quả nhất. Nhờ vậy, A* không chỉ tìm kiếm có mục tiêu mà còn tối ưu hơn hẳn so với các chiến lược duyệt mù như DFS.

6 Kết luận

Về thuật toán DFS: DFS cho thấy khả năng tìm ra lời giải nhanh chóng đối với các bản đồ Sokoban đơn giản và nông. Tuy nhiên, do tính chất tìm kiếm mù quáng và không có cơ chế loại bỏ các đường đi kém hiệu quả, DFS thường gặp vấn đề và không thể giải quyết được các cấp độ phức tạp hơn trong thời gian chấp nhận được. Hơn nữa, DFS không đảm bảo tính tối ưu của lời giải (số lần đẩy hộp nhỏ nhất).

Về thuật toán A*: Với việc áp dụng hàm heuristic hợp lệ, thuật toán A* đã chứng minh khả năng tìm kiếm lời giải tối ưu trong hầu hết các cấp độ. Mặc dù A* thường cần nhiều thời gian và bộ nhớ hơn so với DFS trong giai đoạn đầu, nhưng nhờ cơ chế tìm kiếm thông minh ($f(n) = g(n) + h(n)$), nó đã vượt qua DFS một cách rõ rệt khi đối mặt với các bản đồ phức tạp và sâu.

Như vậy, A* là lựa chọn hiệu quả hơn cho Sokoban vì tính tối ưu và khả năng xử lý không gian trạng thái lớn, miễn là có một hàm heuristic đủ mạnh.

7 Tài liệu tham khảo

Tài liệu

- [1] *Depth-first search*, https://en.wikipedia.org/wiki/Depth-first_search
- [2] *A* algorithm*, https://en.wikipedia.org/wiki/A*_search_algorithm
- [3] *Sokoban Online Puzzle Browser Game*, <https://ksokoban.online/>
- [4] *SOKOBAN HCMUT*, https://github.com/henry1599/Sokoban_AI_Solver_Basic